Vapor-Compression Refrigeration Cycle Analysis

CBE 20260 / Thermodynamics I

This interactive notebook solves a standard vapor-compression refrigeration cycle. You can select different working fluids (refrigerants) to see how their thermodynamic properties impact the coefficient of performance (COP) and the cooling load.

Assumptions: 1. The evaporator and condenser operate at constant pressure. 2. The refrigerant leaves the evaporator as a saturated vapor ($x = 1$). 3. The refrigerant leaves the condenser as a saturated liquid ($x = 0$). 4. The expansion valve is an isenthalpic throttle ($\Delta H = 0$). 5. The compressor has an adjustable isentropic efficiency (η_c).

```
#
 ↪  ================================================================================
# LECTURE 22: Vapor-Compression Refrigeration Cycle Analysis
# Script: vapor-compression-cycle.ipynb
# Author: Edward Maginn, CBE 20260
#
# Description:
# This notebook performs a thermodynamic analysis of a standard
 ↪  vapor-compression
# refrigeration cycle. It calculates the states, work,
# heats, and COP of the cycle based on user inputs for the
 ↪  pressure of the
# evaporator and condenser (or the temperatures, depending on
 ↪  the mode selected).
# The efficiency of the compressor is specified by the user.
# The notebook uses CoolProp to obtain fluid properties and
 ↪  ipywidgets to create
# an interactive interface for exploring how different operating
 ↪  conditions and
# refrigerants affect cycle performance.
#
# Note that the critical point of the refrigerant is a hard
 ↪  limit for the cycle.
# If you exceed it, CoolProp will throw an error, which is
 ↪  caught and displayed
# as a user-friendly message.
#
 ↪  ================================================================================
```

```
#
 ↪  ============================================================================
# Install necessary packages if you don't have them (using quiet
 ↪  mode to
# suppress output)
!pip install -q CoolProp ipywidgets
import CoolProp.CoolProp as CP
import ipywidgets as widgets
from IPython.display import display, clear_output

def solve_vcc(fluid, mode, T_evap_C, T_cond_C, P_evap_bar,
 ↪  P_cond_bar, eta):
    """
    Calculates the states and performance of a standard
 ↪  vapor-compression
    refrigeration cycle.
    """
    try:
        # --- 1. Determine Pressures and Temperatures ---
        if mode == 'Temperatures (°C)':
            T_evap = T_evap_C + 273.15
            T_cond = T_cond_C + 273.15

            if T_evap >= T_cond:
                print("Error: Evaporator temperature must be
                 ↪  lower than Condenser temperature.")
                return

            # Find saturation pressures
            P_evap = CP.PropsSI('P', 'T', T_evap, 'Q', 1, fluid)
            P_cond = CP.PropsSI('P', 'T', T_cond, 'Q', 0, fluid)

        else:
            P_evap = P_evap_bar * 1e5
            P_cond = P_cond_bar * 1e5

            if P_evap >= P_cond:
                print("Error: Evaporator pressure must be lower
                 ↪  than Condenser pressure.")
                return

            # Find saturation temperatures
            T_evap = CP.PropsSI('T', 'P', P_evap, 'Q', 1, fluid)
            T_cond = CP.PropsSI('T', 'P', P_cond, 'Q', 0, fluid)

        # --- 2. Calculate State 1 (Evaporator Out / Compressor
         ↪  In) ---
        # Saturated Vapor
```

```
        H1 = CP.PropsSI('H', 'P', P_evap, 'Q', 1, fluid) / 1000
↪  # kJ/kg
        S1 = CP.PropsSI('S', 'P', P_evap, 'Q', 1, fluid) / 1000
↪  # kJ/kg-K

        # --- 3. Calculate State 2 (Compressor Out / Condenser
         ↪  In) ---
        # Ideal Isentropic Compression
        H2_ideal = CP.PropsSI('H', 'P', P_cond, 'S', S1 * 1000,
↪  fluid) / 1000
        W_c_ideal = H2_ideal - H1

        # Actual Compression
        W_c = W_c_ideal / eta
        H2 = H1 + W_c
        T2 = CP.PropsSI('T', 'P', P_cond, 'H', H2 * 1000, fluid)

        # --- 4. Calculate State 3 (Condenser Out / Valve In)
         ↪  ---
        # Saturated Liquid
        H3 = CP.PropsSI('H', 'P', P_cond, 'Q', 0, fluid) / 1000

        # --- 5. Calculate State 4 (Valve Out / Evaporator In)
         ↪  ---
        # Isenthalpic Throttling
        H4 = H3
        x4 = CP.PropsSI('Q', 'P', P_evap, 'H', H4 * 1000, fluid)

        # --- 6. Cycle Performance Calculations ---
        q_in = H1 - H4          # Cooling Load (Heat absorbed in
↪  Evaporator)
        q_out = H3 - H2         # Heat rejected in Condenser
↪  (IUPAC: negative)
        COP = q_in / W_c        # Coefficient of Performance

        # --- 7. Print Results ---
        print("=" * 60)
        print(f"VAPOR-COMPRESSION CYCLE ANALYSIS: {fluid}")
        print("=" * 60)
        print(f"Evaporator: {P_evap/1e5:.2f} bar |
         ↪  {T_evap-273.15:.2f} °C")
        print(f"Condenser:  {P_cond/1e5:.2f} bar |
         ↪  {T_cond-273.15:.2f} °C")
        print("-" * 60)
        print(f"State 1 (Comp In)  : H1 = {H1:>6.2f} kJ/kg | S1
         ↪  = {S1:.4f} kJ/kg-K")
        print(f"State 2 (Comp Out) : H2 = {H2:>6.2f} kJ/kg | T2
         ↪  = {T2-273.15:.2f} °C")
```

```
        print(f"State 3 (Valve In) : H3 = {H3:>6.2f} kJ/kg |
         ↪  Saturated Liquid")
        print(f"State 4 (Valve Out): H4 = {H4:>6.2f} kJ/kg |
         ↪  Quality (x) = {x4:.3f}")
        print("-" * 60)
        print(f"Cooling Load (q_in):       {q_in:.2f} kJ/kg")
        print(f"Compressor Work (W_c):     {W_c:.2f} kJ/kg")
        print(f"Heat Rejected (q_out):    {q_out:.2f} kJ/kg")
        print("-" * 60)
        print(f"COEFFICIENT OF PERFORMANCE (COP): {COP:.2f}")
        print("=" * 60)

    except ValueError as e:
        print("\n--- FLUID PROPERTY ERROR ---")
        print("The selected conditions are invalid for this
         ↪  fluid.")
        print("You may have exceeded the critical point (e.g.,
         ↪  condensing CO2 above ~31°C).")
        print(f"CoolProp Error: {e}")
        print("Try adjusting the temperatures/pressures or
         ↪  selecting a different fluid.")

# --- CREATE INTERACTIVE WIDGETS ---
style = {'description_width': 'initial'}

# Fluid & Mode Selection
dropdown_fluid = widgets.Dropdown(options=['R134a', 'Ammonia',
 ↪  'CO2', 'R22', 'R410A', 'IsoButane'], value='R134a',
 ↪  description='Refrigerant:', style=style)
dropdown_mode = widgets.Dropdown(options=['Temperatures (°C)',
 ↪  'Pressures (bar)'], value='Temperatures (°C)',
 ↪  description='Input Mode:', style=style)

# Temperature Sliders
slider_T_evap = widgets.FloatSlider(value=-10.0, min=-40.0,
 ↪  max=15.0, step=1.0, description='Evap Temp (°C):',
 ↪  style=style)
slider_T_cond = widgets.FloatSlider(value=30.0, min=10.0,
 ↪  max=60.0, step=1.0, description='Cond Temp (°C):',
 ↪  style=style)
box_T = widgets.VBox([slider_T_evap, slider_T_cond])

# Pressure Sliders (Hidden by default)
slider_P_evap = widgets.FloatSlider(value=2.0, min=0.5,
 ↪  max=15.0, step=0.5, description='Evap Pressure (bar):',
 ↪  style=style)
slider_P_cond = widgets.FloatSlider(value=10.0, min=5.0,
 ↪  max=60.0, step=0.5, description='Cond Pressure (bar):',
 ↪  style=style)
```

```
box_P = widgets.VBox([slider_P_evap, slider_P_cond])
box_P.layout.display = 'none' # Hide initially

slider_eta = widgets.FloatSlider(value=0.75, min=0.6, max=1.0,
 ↪  step=0.01, description='Compressor Efficiency (η):',
 ↪  style=style)

# Dynamic UI toggling based on Mode selection
def on_mode_change(change):
    if change['new'] == 'Temperatures (°C)':
        box_T.layout.display = 'flex'
        box_P.layout.display = 'none'
    else:
        box_T.layout.display = 'none'
        box_P.layout.display = 'flex'

dropdown_mode.observe(on_mode_change, names='value')

# Group everything
ui = widgets.VBox([dropdown_fluid, dropdown_mode, box_T, box_P,
 ↪  slider_eta])

# Connect to function
out = widgets.interactive_output(solve_vcc, {
    'fluid': dropdown_fluid,
    'mode': dropdown_mode,
    'T_evap_C': slider_T_evap,
    'T_cond_C': slider_T_cond,
    'P_evap_bar': slider_P_evap,
    'P_cond_bar': slider_P_cond,
    'eta': slider_eta
})

# Display
display(ui, out)
```

```
VBox(children=(Dropdown(description='Refrigerant:', options=('R134a', 'Ammo
```

```
Output()
```